ECU simulator (`ecusim`)

The genset ECU's real firmware, running on your computer against a model of the engine, generator and fuel rail. You drive it from a browser: start the engine, change the load, break a sensor, and watch speed, rail pressure, the fault codes (DM1) and the CAN traffic react. Scripts (`.seq` files) run whole test scenarios and check the results.

This is mode 1 of three: firmware only. The design is in docs/spec.md.

On Windows: getting started

1. Download the latest `ecusim-windows` from the repository's **Actions** tab: open the newest green **simulator** run, then **Artifacts**. Unzip it.
2. Double-click `ecusim-windows.exe` .
3. Windows may say "Windows protected your PC", because the program isn't signed. Click **More info**, then **Run anyway**.
4. A black window opens and says where it is running, and your browser opens the simulator. Keep the black window open; closing it stops the simulator.

Nothing to install. Run folders are written into a `runs` folder next to where you started it.

On a Mac it is the same, with `ecusim-macos` . The first time, right-click it and choose **Open** (it isn't notarised). On Linux, use `ecusim-linux` .

Using it

Live. Start cranks the engine until the firmware reaches RUN. Then move the **Load** slider, flip the droop switch, or pick a fault and **Apply** it. **Stop (ignition off)** stops the engine. The speed buttons run at 1×, 2× or 10× real time, or as fast as the computer goes (**max**). **Stop** in the top bar ends the run and writes its folder. **Reset** throws it away.

Scripts. Choose one from the library and **Load** it, or **Open...** a `.seq` file, or drag one onto the editor. Then **Run script**. Errors are listed with their line number; click one to jump to the line. **Save as...** downloads what is in the editor. `.seq` files in a `scenarios` folder next to where you started `ecusim` appear in the library too.

When a script ends, the charts show the whole run and the report lists each check as PASS or FAIL with the value found. The **Runs** list opens any earlier run. The page's address then ends in `#run=<folder>` , which you can bookmark.

Firmware. `ecusim` has the firmware built in. To try another build, **Load firmware…** and pick `ecu_fw.dll` (or `.dylib`, `.so`). It is refused if it was built for a different interface (ABI) and the reason is shown. **Only load firmware you trust: it runs on your computer with your rights.** Each run's folder records which firmware made it.

Writing scripts

One line each; `#` starts a comment; times are seconds of simulated time and never go backwards. `end` is required and comes last.

```
# load step 0 → 20 kW
0    start
8    load 20
13   end
expect rpm min 1350.. over 8..13
expect rpm mean 1485..1515 over 12.9..13
expect never dm1 190/18
```

Action	Meaning
`start`	starter on until the firmware reaches RUN, then off
`starter on / off`	the starter by hand
`load <kW>`	electrical load
`ignition on / off`	ignition input
`ignition bounce <ms>`	off for that long, then on
`droop on / off`	droop switch
`sensor <ch> open` / `short` / `short5v` / `stuck` / `clear`	`ch` is `clt oil rail boost egr 5v`. Open reads 5 V on the coolant sensor (pull-up), 0 V on the others
`sensor <ch> volts <V> / offset <V>`	a forced reading, or the model's plus an offset
`crank lost` / `crank restore`	no crank signal / back to normal
`crank drop <n> / crank extra <n>`	the next n teeth missing / n spurious teeth
`cam lost` / `cam restore`	no cam signal / back
`fuel stuck <mg> / fuel stuck <mg> cyl <1-3> / fuel free`	fuel regardless of the firmware (a runaway, or one stuck injector)
`leak <x>`	the rail leaks x times normal
`request dm2 / dm3 / dm4 / hours`	a J1939 request from a service tool
`power cycle`	power off and on: the firmware restarts from its saved memory
`tear next save`	the next memory save is cut halfway, as by a power loss

Check	Passes when
`expect <signal> <mean|min|max> <lo>..<hi> over <t0>..<t1>`	the value over that window is inside the band (`lo..` or `..hi` leave one end open)
`expect <signal> mean change <lo>..<hi> from <a0>..<a1> to <b0>..<b1>`	the mean moves by that much between the windows
`expect dm1 <spn>/<fmi> within <s> of <t>`	a DM1 carrying it is sent within s seconds after t

Check	Passes when
`expect never dm1 <spn>/<fmi>`	it never appears
`expect dm1 clear over <t0>..<t1>`	every DM1 in the window is empty
`expect lamp amber / red+protect / ... with <spn>/<fmi>`	the first DM1 carrying it lights exactly those lamps
`expect trip within <ms> ms of <t>` / `expect no trip`	SHUTDOWN within that time / never
`expect state RUN at end`	the state when the run stops (`OFF WAIT CRANK RUN SHUTDOWN`)
`expect stored <spn>/<fmi>` / `expect not stored ...`	in the fault memory at the end
`expect dm2 <spn>/<fmi> after <t>` / `expect dm2 empty after <t>`	the first DM2 answer after t

Signals: `rpm` (the engine's), `ecu_rpm` (the firmware's reading), `rail`, `rail_err` (rail over the firmware's setpoint, minus 1), `duty`, `fuel`, `clt`, `oil`, `load`. An FMI of `*` matches any.

The library's `sil/` scripts are the firmware's 16 test scenarios. `faults/` has one demonstration per fault.

Run folders

Every run writes `runs/<date>-<time>-<name>/` (the time is UTC):

File	Content
`trace.csv`	one row per simulated millisecond
`can.csv`	every CAN frame, sent and received
`events.txt`	everything done, scripted or by hand, as a script: it replays
`report.txt`	each check and what was found
`firmware.txt`	which firmware: built in, or the file's name and SHA-256

From the command line

```
ecusim                          open the browser UI (port 8025, or the next free one)
ecusim --port 9000 --share      let other computers on the network watch; firmware uploads are r
ecusim run scenarios/sil/*.seq  run scripts headless, as fast as possible
ecusim run my.seq --fw ecu_fw.dll --out results
```

`ecusim run` exits 0 if every check passes, 1 if any fails, and 2 on a script or firmware error.

Building from source

Rust 1.80 or newer (https://rustup.rs). On Windows that needs the Visual Studio C++ Build Tools, which rustup offers to install. They also compile the firmware.

```
cd simulator
cargo build --release -p ecusim     # target/release/ecusim(.exe)
cargo test --workspace              # everything, including the 16 scenarios
cargo run --release -p fwbuild      # the firmware as build/ecu_fw.dll / .dylib / .so
```

`fwbuild --core DIR --out FILE -D NAME=VALUE` builds another copy of the firmware, such as a branch you are trying.

The crates: `plant` is the engine model, ported from `firmware/sim/engine.py` and checked against its traces. `sim` is the firmware binding, the loop, scripts and checks. `ecusim` is the program and its page (`crates/ecusim/ui/`). `fwbuild` builds the firmware library.

Changing `ecu_io` or an entry point in `firmware/core/ecu.h`? Bump `ECU_ABI_VERSION` there and `ABI_VERSION` in `crates/sim/src/fw.rs`, and copy the struct change into `EcuIo`. The `io_size_matches` test catches a missed field.